making a connection between LLCS and generic on/off server, changing occupancy state will automatically change on/off state. In other words, as you walk into the room, the sensor will notice your presence and the light will switch on. We can build relationships between these models to achieve all sorts of useful things.

In place of an occupancy sensor, imagine having an ambient light sensor. This is a common requirement in smart buildings. With the ambient light sensor, light levels can be sampled and as changes occur, messages can be sent to report ambient light levels to LLCS model. It has a state binding with LLS. When the sensor detects brightness from an external source (bright sunlight), then the light will get dim or may get completely turned off.

Conversely, when it gets darker, the light will get brighter. A time server can also be included in the model to schedule time changes.

Smart lights consist of some hardware, possibly some sensors integrated into that same unit. These include a microcontroller (MCU) and Bluetooth Mesh communications component that can communicate with other devices in the mesh network. At the top of this stack, there is application software that controls the lights (switching on/off, changing colours, etc). For multiple lights, multiple applications can be implemented on that same MCU. This way every single light can become a beacon.

Bluetooth beacons are becoming increasingly common. These are Bluetooth transmitters that broadcast a unique ID. If you are walking with a smartphone with some software on it, and it receives that broadcast with a unique ID, it can translate that ID into knowledge of a place or an object. Bluetooth beacons are often used to give localised information when you are in proximity of something, or to give indoor navigation capabilities. So, in a really large building like an airport, you can deploy an indoor navigation system because you have the infrastructure.

Ways of creating a mesh network

There are two distinct ways of creating a mesh network. One involves routing, which is figuring out how to get data from point A to B via intermediate points. There are algorithms that determine routes. However, routing gives rise to a single point of failure in the network, and that is not acceptable. Routing is an efficient way of handling data, but it is not sufficiently reliable.

Flooding, on the other hand, is at the other end of the spectrum and involves no routing whatsoever. A message gets broadcast, and every device that receives it broadcasts it again. So, the message spreads across the whole network and everything receives everything. It gets through in a reliable way, given the amount of re-transmission that goes on. But it is not very efficient.

Bluetooth Mesh uses something between the two, called managed flooding, which uses some fundamentals of flooding—routing is not done to avoid single points of failure. But there are various strategies of doing flooding that make it efficient. This gives the best of both the worlds. You can control how many hops a message will do. There are heartbeat messages flowing across the network, so devices can inform each other about where they are or how far they are. This way the network can learn about itself and its topology, and use that to optimise the way it sends data, how far messages will go and so on.

Devices that are part of the provisioned network are called nodes. Regardless of the type of product, nodes can fulfil a variety of network functions, whether it is a light, light switch or thermostat. In relay nodes, messages get relayed (hopping through the network) only by nodes that are acting as relay nodes. This is a software feature, and not all devices or nodes in the network are relays. You can decide when you are designing and implementing your network, as to which devices will provide that service.

Friendship in Bluetooth network works like this. In any network, particularly a smart building, there are devices that are power-rich such as lights. These have limitless power because they are connected to the grid. And then there are low-power nodes, such as sensors, which are embedded on a surface and, hence, their batteries cannot be changed every time. Managing their power consumption is absolutely critical.

So, if a low-power node is equipped with a temperature sensor and is supposed to report temperature only when it goes below a certain limit then it is a very efficient sensor because it is going to send one byte of data every three months, or something like that. Hence, there is no problem. But, if one of the requirements is that you must be able to change those thresholds by sending a message to the sensor whenever you want to, there will be an efficiency problem because the sensor needs to use the radio to listen for messages a lot of the time.

Friendship involves low-power nodes discovering devices that act like friends. It is a software feature that can send out messages to any other device, and these two devices work together. Messages that get sent to the low-power sensor actually get stored by the friend, which will accrue messages addressed to the sensor until the sensor wakes up and receives them. So, devices can discover and agree to help each other.

The last network role a node can perform is called proxy, which is really important because proxy lets any compatible Bluetooth Low